MINI PROJECT REPORT

EDGE DETECTION AND IMAGE ANALYSIS TOOL

Submitted in partial fulfilment of the academic requirements for
Image Processing and Analysis (IPA)

Submitted to
Dr. Sricheta Parui

Submitted by	Amarindar Singh
Roll Number	24155749
Programme	B.Tech – Computer Science and Engineering (AI & ML)
University	KIIT University, Bhubaneswar
Academic Year	2026

Department of Computer Science and Engineering
KIIT University, Bhubaneswar, Odisha

ABSTRACT

This mini project presents a Python-based Edge Detection and Image Analysis Tool that demonstrates commonly used classical image-processing operators. The application accepts a digital image, converts it into grayscale, and applies Roberts, Prewitt, Sobel, Laplacian and Canny edge-detection methods. The resulting images are displayed together for visual comparison. The project illustrates how image gradients and intensity changes can be used to identify object boundaries, shapes and structural details. It is intended as an educational implementation of concepts from Image Processing and Analysis, rather than a replacement for specialist computer-vision systems.

Keywords: Digital image processing, edge detection, grayscale, gradient, Roberts, Prewitt, Sobel, Laplacian, Canny, OpenCV.

TABLE OF CONTENTS

No.	Section	Page
1.	Introduction and Problem Statement	3
2.	Objectives, Importance and Novelty	3
3.	Tools, Requirements and Methodology	4
4.	Edge Detection Techniques	5
5.	Implementation and Results	6
6.	Conclusion, Limitations and Future Scope	7
7.	References	8

Note: The page numbers correspond to this report layout. Update them if the report is edited or repaginated.

1. INTRODUCTION

Digital images are used in areas such as medical imaging, satellite observation, industrial inspection, photography and security. An image contains objects and regions separated by changes in brightness or colour. Edge detection is an important image-processing operation that identifies locations where image intensity changes sharply. These locations often correspond to object boundaries, lines, corners or surface transitions.

This project implements five classical edge-detection techniques using Python, OpenCV and NumPy. A user selects an image from the computer. The program converts it to grayscale and processes it with each operator. Matplotlib presents the original image and processed outputs in a single comparison layout.

1.1 Problem Statement

Different edge detectors respond differently to image details, noise and intensity changes. It can be difficult for a learner to understand these differences by studying formulas alone. The project addresses this learning problem by applying several methods to the same input image and displaying their outputs side by side.

2. OBJECTIVES

- To understand the role of edge detection in digital image processing.
- To convert a colour image into grayscale for intensity-based analysis.
- To implement Roberts, Prewitt, Sobel, Laplacian and Canny methods.
- To observe and compare the edge maps produced by each method.
- To create a simple, reproducible program using open-source Python libraries.

2.1 Importance and Applications

Edge information reduces an image to important structural boundaries and can support later tasks such as object recognition, image segmentation, measurement and feature extraction. Edge detection is used in road-lane analysis, document processing, medical-image analysis, industrial quality inspection, mapping and scene understanding. The project provides a practical introduction to this foundational operation.

2.2 Novelty / Project Contribution

The project does not claim to invent a new edge-detection algorithm. Its contribution is an integrated educational tool that applies five established operators to the same user-selected image and presents the outputs in a consistent comparison view. This makes differences in edge response easier to observe and supports hands-on learning, experimentation and discussion of algorithm behaviour.

3. TOOLS, REQUIREMENTS AND METHODOLOGY

3.1 Software Requirements

Component	Purpose
Python 3	Programming language used to implement the application.
OpenCV (cv2)	Image loading, grayscale conversion and filtering operations.
NumPy	Creation of convolution kernels and numerical calculations.
Matplotlib	Displaying and saving the comparison of processed images.
Tkinter	Opening a file-selection dialog to choose an input image.
Visual Studio Code	Code editing and project execution environment.

3.2 Hardware Requirements

A standard laptop or desktop computer capable of running Python 3 and the listed libraries is sufficient. No GPU or specialised hardware is required.

3.3 Methodology / Workflow

Step 1: The user selects an image using a file dialog.
Step 2: OpenCV reads the image from the selected path.
Step 3: The image is converted from BGR colour format to grayscale.
Step 4: The five edge-detection methods are applied to the grayscale image.
Step 5: Matplotlib displays the original, grayscale and processed images with labels.
Step 6: The comparison figure is saved as *edge_detection_results.png*.

3.4 Basic Processing Concepts

A grayscale image represents each pixel using an intensity value, typically from 0 (black) to 255 (white) for an 8-bit image. Edge detectors examine changes in intensity between neighbouring pixels. Convolution applies a small matrix called a kernel to local image regions. The kernel responses estimate directional or second-order intensity changes, which are used to highlight edges.

4. EDGE DETECTION TECHNIQUES

4.1 Roberts Operator

The Roberts operator uses two 2 × 2 kernels to estimate diagonal intensity differences. It is computationally simple and responds to fine diagonal edges. Because its kernels are small, it can be sensitive to noise and may produce weaker responses in some regions.

4.2 Prewitt Operator

The Prewitt operator uses 3 × 3 horizontal and vertical kernels to estimate image gradients. The two directional responses are combined to obtain an edge-strength image. It is straightforward to implement and can identify horizontal and vertical boundaries.

4.3 Sobel Operator

The Sobel operator also uses 3 × 3 kernels to estimate horizontal and vertical gradients. Its kernel weighting gives greater emphasis to the central row or column, providing a degree of smoothing while calculating the gradient. The gradient magnitude is commonly calculated as the square root of the sum of squared directional responses.

4.4 Laplacian Operator

The Laplacian is a second-order derivative operator. It responds to rapid changes in intensity in multiple directions and can highlight fine boundaries. Since second derivatives can amplify noise, smoothing is often applied before Laplacian filtering in practical systems.

4.5 Canny Edge Detection

Canny is a multi-stage edge-detection method. Its typical stages are Gaussian smoothing, gradient calculation, non-maximum suppression, double-threshold classification and edge tracking by hysteresis. In this project, OpenCV's Canny function is used with lower and upper thresholds of 100 and 200.

4.6 Comparison

Method	Main idea	Typical observation
Roberts	2 × 2 diagonal differences	Simple, fine responses; noise-sensitive.
Prewitt	3 × 3 first derivatives	Highlights horizontal and vertical edges.
Sobel	Weighted 3 × 3 gradients	Clear gradient edges with mild smoothing.
Laplacian	Second derivative	Responds to rapid intensity changes.
Canny	Multi-stage edge detector	Thin, connected edges after thresholding.

5. IMPLEMENTATION AND RESULTS

5.1 Implementation Overview

The program is implemented in a single Python file named *main.py*. The file-selection dialog allows the user to choose a JPG, JPEG or PNG image. OpenCV loads the selected image, converts it to grayscale and performs the required filtering operations. NumPy is used to define the Roberts and Prewitt kernels and combine directional responses. Matplotlib arranges the seven views in a labelled comparison figure and saves the result as a PNG file.

5.2 Experimental Output

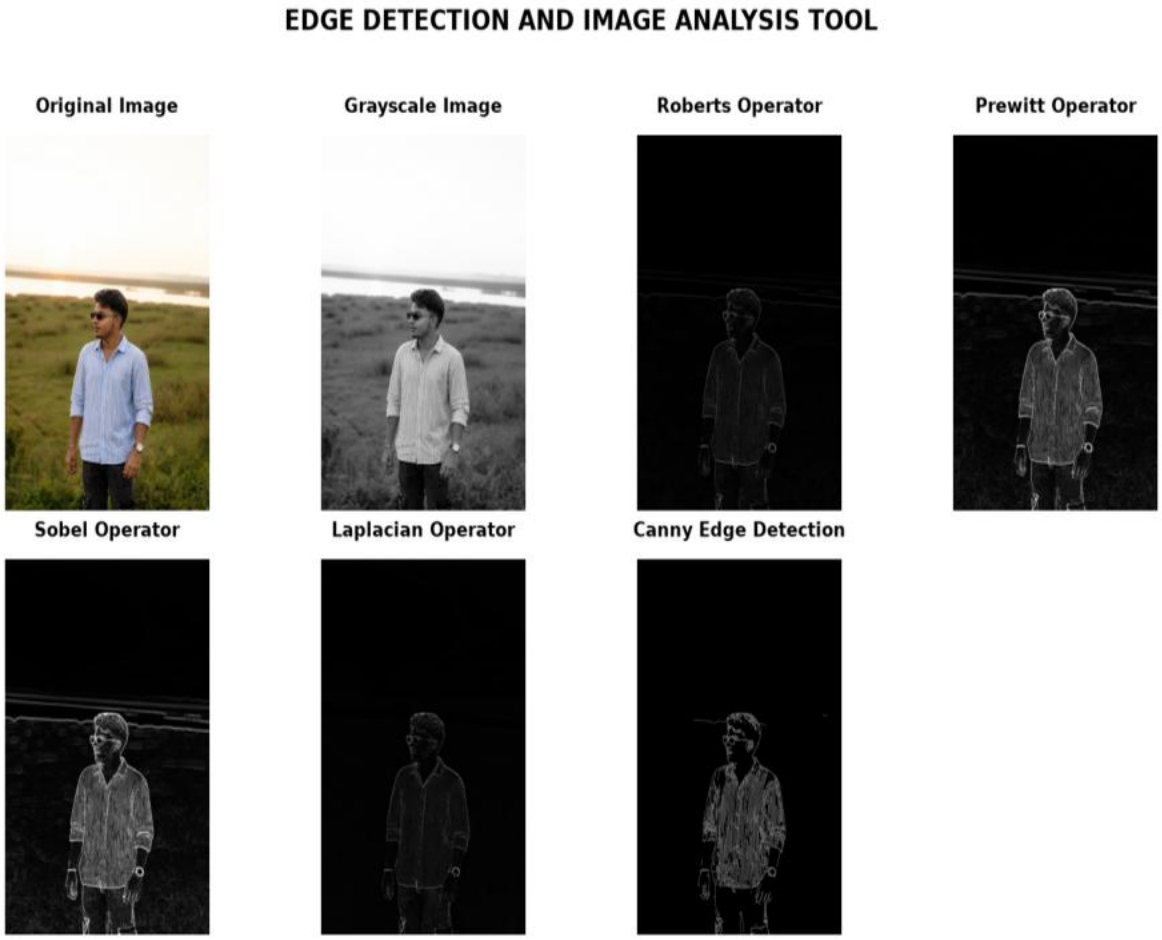


Figure 1. Output of the Edge Detection and Image Analysis Tool showing the original image, grayscale conversion and five edge-detection results.

5.3 Result Discussion

The program successfully generates an output for each implemented method. The Roberts result shows local diagonal intensity changes, while Prewitt and Sobel reveal broader directional boundaries. The Laplacian response highlights rapid intensity variation, and Canny produces a more selective edge map through its multi-stage process. The appearance and strength of edges depend on image content, contrast, noise and parameter settings. Therefore, the methods should be compared qualitatively rather than assuming that one output is always best.

6. CONCLUSION, LIMITATIONS AND FUTURE SCOPE

6.1 Conclusion

The Edge Detection and Image Analysis Tool demonstrates five fundamental edge-detection techniques using a simple Python application. It accepts a user-selected image, converts it to grayscale, applies Roberts, Prewitt, Sobel, Laplacian and Canny methods, and presents the results together for comparison. The implementation helped demonstrate how first-order gradients, second-order derivatives and multi-stage processing can identify image boundaries. The project meets its educational objective of making core Image Processing and Analysis concepts observable through practical experimentation.

6.2 Limitations

- The current program processes one image at a time.
- Edge quality depends on the image's contrast, lighting, texture and noise.
- Canny uses fixed threshold values (100 and 200) in the current implementation.
- The project provides visual comparison and does not calculate a formal accuracy score because no ground-truth edge labels are supplied.

6.3 Future Scope

The tool can be extended with adjustable thresholds, Gaussian pre-filtering, image resizing controls, batch processing and options to save each individual edge map. Further work could include quantitative comparison using labelled edge datasets and additional methods such as LoG or Scharr operators. A graphical interface could also be added if required.

6.4 Learning Outcomes

Through this project, the student practised image loading, colour-space conversion, convolution, gradient computation, edge detection, visualisation, debugging and the use of open-source computer-vision libraries. The project also demonstrates how theoretical operators can be implemented and compared on real images.

PROJECT FILES

The source code, dependency list, output image and usage instructions should be maintained in the associated GitHub repository. The repository link can be added here after it is created.

GitHub Repository: _____

7. REFERENCES

The following resources were used as technical references for the concepts and software libraries discussed in this project:

1. OpenCV Documentation, “Image Filtering” and “Canny Edge Detector.” Available at: https://docs.opencv.org/ (accessed 3 November 2026).

2. OpenCV-Python Tutorials, “Canny Edge Detection.” Available at: https://docs.opencv.org/4.x/da/d22/tutorial_py_canny.html

3. OpenCV-Python Tutorials, “Smoothing Images” and image-processing operations. Available at: https://docs.opencv.org/4.x/d4/d13/tutorial_py_filtering.html

4. NumPy Documentation. Available at: https://numpy.org/doc/

5. Matplotlib Documentation. Available at: https://matplotlib.org/stable/

6. R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed., Pearson, 2018.

7. J. Canny, “A Computational Approach to Edge Detection,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. PAMI-8, no. 6, pp. 679–698, 1986.

ACKNOWLEDGEMENT

I express my sincere gratitude to the faculty of the Image Processing and Analysis course for providing the opportunity to work on this mini project. I also acknowledge the developers and maintainers of Python, OpenCV, NumPy and Matplotlib for making accessible tools available for learning and experimentation.

— END OF REPORT —